



PROJECT REPORT

Signals & Systems

IMAGE AS A 2D SIGNAL: RGB TO EDGE DETECTION USING MATLAB

Group Members

Heba Saqib (02-239232-061)
Mariam Shoaib (02-239232-070)

Electrical Engineering Department
Bahria University, Karachi Campus

FALL 2025

ABSTRACT

This project presents an image as a two-dimensional signal and demonstrates its processing using MATLAB. An RGB image is transformed into grayscale, indexed, and binary forms to simplify representation and highlight structural features. Edge detection is then applied to extract significant boundaries within the image. The methodology illustrates how signal processing concepts are extended to image analysis. Results validate that successive transformations reduce data complexity while preserving essential information, making the approach suitable for practical image-processing applications.

CHAPTER 1

1. INTRODUCTION

1.1 Background

Digital images can be modeled as two-dimensional signals where pixel intensity varies across spatial coordinates. Signal processing techniques traditionally applied to one-dimensional signals can be extended to images for analysis, enhancement, and feature extraction. Image representation and transformation form the foundation of modern applications such as computer vision, medical imaging, and pattern recognition.

1.2 Literature Review

Various studies have shown the effectiveness of grayscale conversion, thresholding, and edge detection for simplifying image data while retaining critical information. Classical edge detection techniques such as Sobel, Prewitt, and Canny operators are widely used due to their robustness and computational efficiency [1]–[3]. MATLAB provides optimized tools for implementing these techniques efficiently.

1.3 Problem Statement

RGB images contain large amounts of redundant color information, making direct processing computationally expensive. The challenge is to transform the image into simpler representations while preserving important features such as object boundaries.

1.4 Aims and Objectives

The objectives of this project are:

- To represent an image as a 2D signal
- To convert an RGB image into grayscale form

- To transform a grayscale image into indexed and binary formats
- To perform edge detection on a binary image
- To analyze the effect of each transformation stage

1.5 Scope of Project

The scope of this project is limited to basic image transformations and edge detection using MATLAB. Advanced filtering and frequency-domain analysis are beyond the current scope but can be explored in future work.

1.6 Environmental Aspects of Project

The project is software-based and does not involve hardware manufacturing or waste generation. Efficient image processing contributes indirectly to reduced computational energy usage in real-world systems.

CHAPTER 2

2. DESIGN AND METHODOLOGY

2.1 Equipment / Materials Used

- Personal Computer or Laptop
- MATLAB Software
- MATLAB Image Processing Toolbox
- Sample RGB Image (.jpg / .png)

2.2 Circuit Diagram

Not applicable as this project is software-based.

2.3 Implementation

The implementation follows a step-by-step pipeline. First, an RGB image is loaded into MATLAB. The image is converted into grayscale to reduce dimensionality. The grayscale image is then transformed into an indexed image using a grayscale colormap. Thresholding is applied to obtain a binary image, and finally, edge detection is performed using the Canny method.

CHAPTER 3

3. RESULTS AND ANALYSIS

3.1 Simulation / Experimental Results

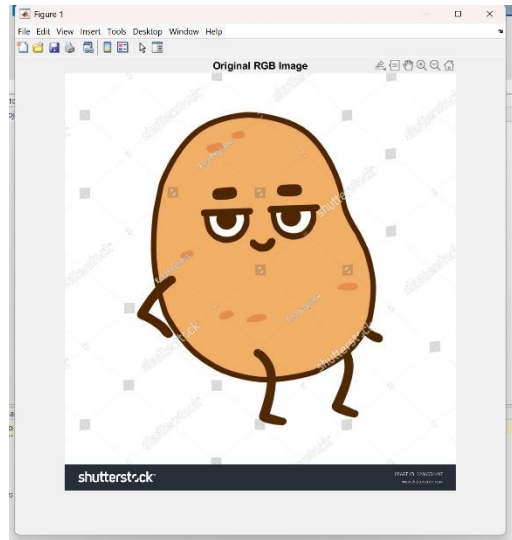


Figure 31: Original RGB Image

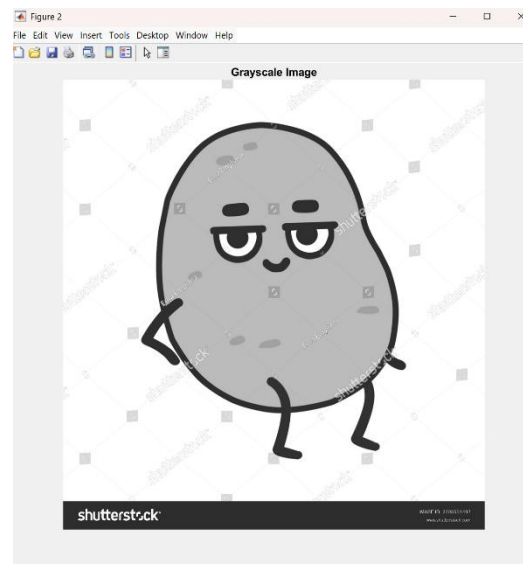


Figure 32: Grayscale Image

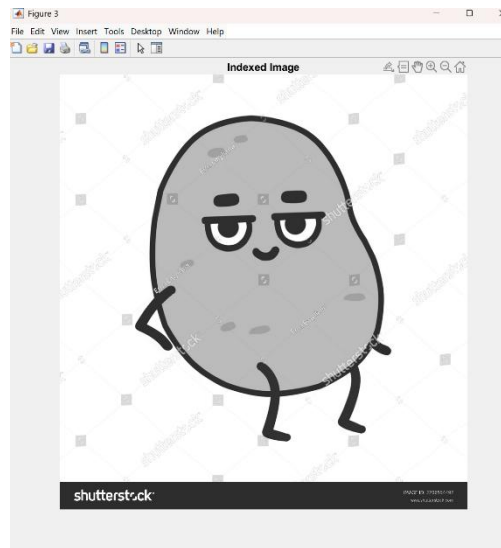


Figure 33: Indexed Image

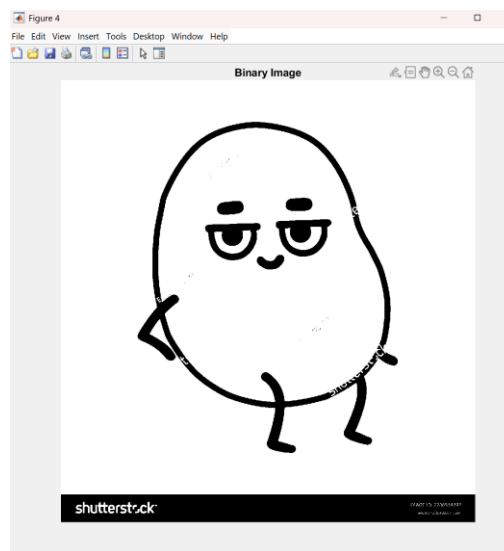


Figure 34: Binary Image

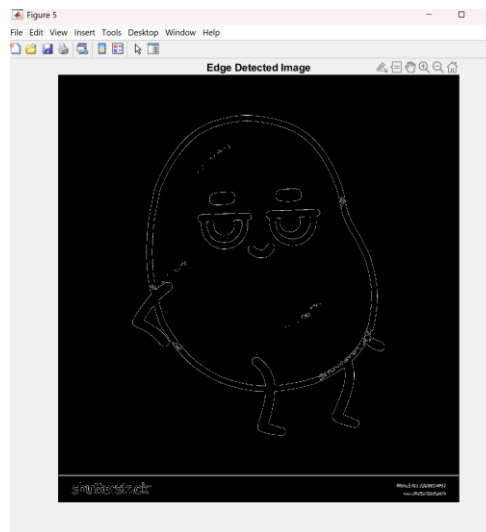


Figure 35: Edge Detected Image

3.2 Data in Tabular / Graphical Form

Not applicable, as the results are visual in nature.

3.3 Comparison with Expected Outcomes

The grayscale and indexed images appear visually similar, as expected, due to the use of a grayscale colormap. The binary image successfully separates foreground and background, and the edge detection output highlights object boundaries clearly.

3.4 Discussion and Interpretation of Results

Each transformation stage reduces image complexity while preserving essential features. Edge detection effectively extracts high-frequency components corresponding to object boundaries.

3.5 Identification of Limitations

- Sensitive to threshold selection
- Performance depends on image quality
- Limited to spatial-domain processing

CHAPTER 4

4. CONCLUSIONS AND RECOMMENDATIONS

The project successfully demonstrated image processing as a 2D signal using MATLAB. All objectives were achieved, and the transformations provided meaningful insight into image representation. Future work may include frequency-domain analysis, noise reduction techniques, and comparison of different edge detectors.

REFERENCES

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Pearson, 2018.
- [2] J. Canny, "A Computational Approach to Edge Detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986.
- [3] MATLAB Documentation, Image Processing Toolbox.

APPENDICES

APPENDIX A: Computer Program Listing (Code)

```
clc;
clear;
close all;

% Read RGB image (change file name if needed)
[file, path] = uigetfile({'*.jpg;*.png;*.bmp', 'Image Files'});
rgb = imread(fullfile(path, file));

% Display original RGB image
figure;
imshow(rgb);
title('Original RGB Image');

% Convert RGB to Grayscale
gray = rgb2gray(rgb);
figure;
imshow(gray);
title('Grayscale Image');

% Convert Grayscale to Indexed Image
[indexed, cmap] = gray2ind(gray, 256);
figure;
imshow(indexed, cmap);
title('Indexed Image');
```

```
% Convert Indexed Image to Binary Image
binary = imbinarize(indexed);
figure;
imshow(binary);
title('Binary Image');

% Edge Detection from Binary Image
edges = edge(binary, 'Canny');
figure;
imshow(edges);
title('Edge Detected Image');

% Display all results together
figure;
subplot(2,3,1), imshow(rgb), title('RGB Image');
subplot(2,3,2), imshow(gray), title('Grayscale');
subplot(2,3,3), imshow(indexed, cmap), title('Indexed');
subplot(2,3,4), imshow(binary), title('Binary');
subplot(2,3,5), imshow(edges), title('Edges');
```

APPENDIX B: Screenshots / Graphs

